



UNIVERSITÀ DEGLI STUDI
DI GENOVA

Computer Games

PULSE SAVER (CPR TRAINING GAME)

Name: Fazeel Ahmed

Matricola: S5640427

Group: Single Member

Submitted To:

Professor Marco Raggio

PULSE SAVER

CPR Training Game

Technical Design and Implementation Report

1. Executive Summary

Pulse Saver is an interactive CPR training game developed in Unity. The player performs simulated chest compressions on a patient and receives immediate feedback about compression rhythm, patient health, score, progress, and final outcome. The purpose of the game is to convert CPR timing into a simple, game-like training experience in which the player must maintain a target compression rate of 100-120 compressions per minute.

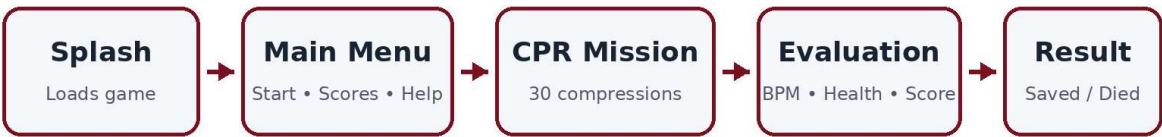
The gameplay scene is fully editable in the Unity Hierarchy. The patient, environment, Canvas, buttons, text elements, progress bars, and result panel are pre-built scene objects rather than runtime-generated GUI objects. This makes the project easier to inspect, modify, demonstrate, and extend.

2. Project Overview

Component	Implementation
Game name	Pulse Saver
Engine	Unity 6.1
Programming language	C#
UI system	Unity UI (uGUI)
Main scenes	Splash, MainMenu, CPRGame
CPR target rate	100-120 BPM
Mission length	30 chest compressions
Success threshold	At least 80% rhythm accuracy and sufficient patient health
Persistent data	JSON score file using Application.persistentDataPath

3. Game Flow

Pulse Saver - Game Flow



The user first sees the splash screen, then enters the main menu. From the main menu the player can start the CPR mission, read instructions, or view saved scores. During the CPR mission, a countdown begins and the player performs compressions using the PUMP CHEST button or the Space key. After 30 compressions, the game calculates the final result and displays whether the patient was saved or died.

4. Scene and Script Architecture

Splash scene: Displays the Pulse Saver branding, loading progress, and automatically opens the main menu.

MainMenu scene: Contains Start Training, How To Play, Scores, Reset Scores, and Quit controls.

CPRGame scene: Contains the patient, training environment, HUD, score, health, BPM indicator, compression control, countdown, and result panel.

CPRGame.cs: Controls the CPR gameplay logic, timing, BPM calculation, patient health, score, streak, result, and patient animation.

SaveManager.cs: Stores the best score, last score, number of attempts, saves, failures, last result, and last-played date in JSON.

MainMenu.cs: Handles menu button actions and displays previously saved score statistics.

SplashController.cs: Updates the splash loading bar and loads the MainMenu scene.

5. CPR Gameplay Logic

When the player starts a mission, the game resets all counters and patient state, shows a 3-2-1-GO countdown, and then enables CPR input. Each valid click or Space key press represents one chest compression.

5.1 Compression Rate (BPM) Calculation

Pulse Saver does not measure an actual heart pulse. Instead, it measures the time interval between two consecutive chest-compression inputs. The interval is converted into compressions per minute using the formula $BPM = 60 / \text{interval}$. A rate between 100 and 120 BPM is treated as correct.

Key code: calculating the CPR compression rate

```
float interval = Time.time - lastCompressionTime;
float bpm = 60f / Mathf.Max(interval, 0.01f);
bool correct = bpm >= minBpm && bpm <= maxBpm;

bpmText.text = "BPM  " + Mathf.RoundToInt(bpm);
```

For example, if the player waits 0.5 seconds between two compressions, the calculated rate is $60 / 0.5 = 120$ BPM. If the interval is 0.6 seconds, the rate is 100 BPM. Therefore, the correct timing window is approximately 0.50 to 0.60 seconds between compressions.

5.2 Correct and Incorrect Compression Feedback

After the BPM is calculated, the game checks whether the value lies inside the accepted range. A correct rhythm increases the correct-compression count, streak, and patient health. An incorrect rhythm resets the streak and decreases patient health. The HUD immediately tells the player whether they are too fast or too slow.

Key code: CPR quality feedback and patient-health update

```
if (correct)
{
    correctCompressions++;
    streak++;
}
```

```

    bestStreak = Mathf.Max(bestStreak, streak);
    health = Mathf.Clamp(health + correctHealthGain, 0, 100);

    feedbackText.text = "PERFECT RHYTHM";
    bpmIndicator.color = Good;
}
else
{
    streak = 0;
    health = Mathf.Clamp(health - wrongHealthLoss, 0, 100);

    feedbackText.text = bpm > maxBpm
        ? "TOO FAST - SLOW DOWN"
        : "TOO SLOW - SPEED UP";

    bpmIndicator.color = Bad;
}

```

5.3 Patient Health System

The patient begins the mission with a configurable health value. In the current configuration, patient health starts at 60. A correct compression interval adds 2 health points, while an incorrect interval removes 5 points. Health is always clamped between 0 and 100. If health reaches 0 before the mission finishes, the patient immediately dies and the mission ends.

Key code: patient health and failure condition

```

health = Mathf.Clamp(health + correctHealthGain, 0, 100);
// or, for a wrong rhythm:
health = Mathf.Clamp(health - wrongHealthLoss, 0, 100);

if (health <= 0)
{
    FinishMission(false);
    return;
}

```

6. Scoring System

Pulse Saver uses a 0-100 score. During gameplay, the live score combines three values: rhythm accuracy, current patient health, and mission progress. This gives the player continuous feedback instead of waiting until the end of the mission.

Key code: live score calculation

```

int measured = Mathf.Max(1, compressionCount - 1);
float accuracy = correctCompressions / (float)measured;
float healthPart = health / 100f;
float progressPart = compressionCount / (float)totalCompressions;

score = Mathf.Clamp(
    Mathf.RoundToInt(
        accuracy * 70f +
        healthPart * 20f +
        progressPart * 10f),
    0,
    100);

```

The weighting gives 70% importance to CPR rhythm accuracy, 20% to patient health, and 10% to completion progress. This means the player cannot obtain a high score simply by finishing the level; maintaining the correct rhythm is the most important factor.

6.1 Final Score

At the end of the mission, the game calculates a final score. The final score uses rhythm accuracy as the main factor, patient health as the second factor, and the best streak as a small bonus.

Key code: final score calculation

```
int accuracyPercent = Mathf.RoundToInt(
    Mathf.Clamp01(correctCompressions / (float)measured) * 100f);

int finalScore = Mathf.Clamp(
    Mathf.RoundToInt(
        accuracyPercent * 0.72f +
        health * 0.20f +
        Mathf.Min(bestStreak, 10) * 0.8f),
    0,
    100);
```

7. Patient Saved / Patient Died Decision

The mission normally finishes after 30 compressions. The first compression cannot be evaluated for rhythm because there is no previous compression interval, so the accuracy denominator uses totalCompressions - 1. The patient is saved only when the measured rhythm accuracy is at least 80% and the patient still has enough health.

Key code: deciding the final patient outcome

```
if (compressionCount >= totalCompressions)
{
    int measured = Mathf.Max(1, totalCompressions - 1);
    float accuracy = correctCompressions / (float)measured;

    bool saved = accuracy >= requiredAccuracy && health >= 35;
    FinishMission(saved);
}
```

8. GUI and Player Feedback

The CPRGame scene uses a fully pre-built Canvas. All important interface elements are visible and editable in the Unity Hierarchy before the game runs. No GUI is generated dynamically during gameplay.

GUI element	Purpose
Status text	Shows Ready, Get Ready, Rescue in Progress, Patient Stabilizing, Patient Critical, Mission Success, or Mission Failed.
BPM text + indicator	Displays current compression BPM and changes color based on CPR rhythm.
Patient Health bar	Shows the simulated condition of the patient.
CPR Progress bar	Shows progress toward 30 compressions.
Score	Displays the current 0-100 game score.
Streak / Combo	Rewards consecutive correct compression intervals.
PUMP CHEST button	Main interactive control for performing a compression.

Countdown	Displays 3, 2, 1, GO before CPR input is enabled.
Result panel	Shows Patient Saved / Patient Died, stars, final score, accuracy, health, and best streak.

9. Chest Compression Animation

Every compression briefly scales and moves the torso to create visible chest movement. A sine curve is used so the chest presses down and then returns smoothly to its original position. The red chest target also pulses continuously to show the player where to perform the compression.

Key code: visual chest-compression animation

```
float t = 1f - Mathf.Clamp01(
    compressionAnimationTimer / CompressionAnimationDuration);
float press = Mathf.Sin(t * Mathf.PI);

Vector3 scale = torsoBaseScale;
scale.y *= Mathf.Lerp(1f, 0.66f, press);
torso.localScale = scale;

Vector3 pos = torsoBasePosition;
pos.y -= 0.05f * press;
torso.localPosition = pos;
```

10. Score Saving Using JSON

The game saves player statistics after each completed mission. Unity's `persistentDataPath` is used so the score file remains available between game sessions. The stored data includes best score, last score, attempts, number of patients saved, failed rescues, last result, and last-played time.

Key code: persistent score storage

```
public static string SavePath =>
    Path.Combine(Application.persistentDataPath, "rescuemate_scores.json");

public static void RecordResult(int score, bool saved)
{
    RescueMateSaveData d = Load();
    d.lastScore = score;
    d.bestScore = Mathf.Max(d.bestScore, score);
    d.attempts++;

    if (saved) d.patientsSaved++;
    else d.failedRescues++;

    Save();
}
```

11. Main Menu and Scene Navigation

The main menu provides access to the training mission, instructions, saved scores, and quit action. Scene navigation uses Unity `SceneManager`. This keeps the flow simple and separates the menu from gameplay.

Key code: scene navigation

```
public void StartTraining()
{
    SceneManager.LoadScene("CPRGame");
}
```

```
public void BackToMenu()
{
    SceneManager.LoadScene("MainMenu");
}
```

12. Editable Scene Design

A major design requirement was that the GUI and game objects must not be generated during runtime. The final structure therefore uses an Editor-only scene builder to create the initial project scenes once. After generation, the objects exist normally in the scene and can be edited through the Inspector and Hierarchy like any standard Unity project.

For example, the CPRGame scene contains separate objects for the Main Camera, Environment, Patient, GameManager, Canvas, EventSystem, and lights. Inside the Canvas, the TopHUD, MissionHUD, ActionDock, CountdownText, ComboText, and ResultPanel are independently editable. The button OnClick events are also connected as persistent Unity events.

13. Testing Scenarios

Test	Expected behavior	Result criteria
Correct rhythm	Keep approximately 0.50-0.60 s between compressions	Green BPM indicator, health increases, score rises
Too fast	Compress with intervals below ~0.50 s	Too Fast message, health decreases, streak resets
Too slow	Compress with intervals above ~0.60 s	Too Slow message, health decreases, streak resets
Patient health reaches 0	Repeated incorrect rhythm	Mission ends immediately with Patient Died
30 compressions with >=80% accuracy	Maintain correct timing for most intervals	Patient Saved if health is at least 35
Score persistence	Complete mission and reopen Scores	Best/last score and attempts remain stored

14. Limitations and Future Improvements

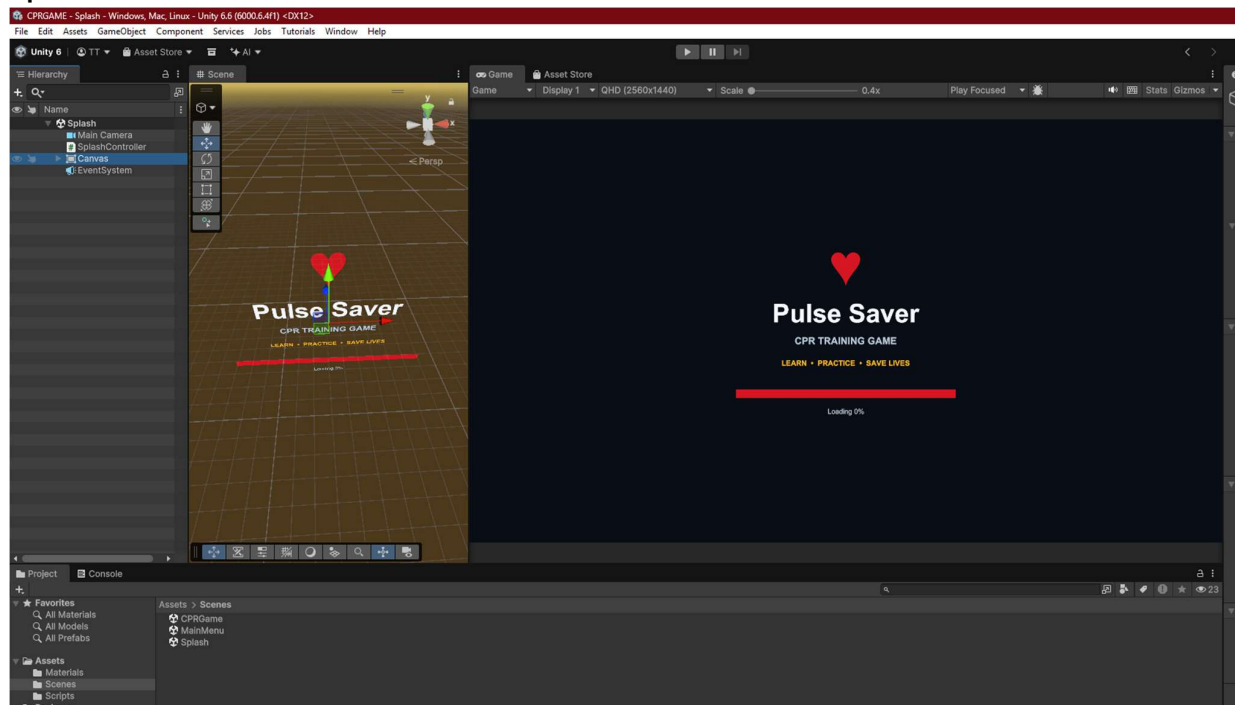
The current project is a game simulation rather than a certified medical training system. It evaluates input timing only; it does not measure real compression depth, hand position, recoil, ventilation quality, or a patient's physiological heartbeat. Future versions could connect a pressure sensor, accelerometer, CPR mannequin, or wearable sensor to provide real measurements.

- Replace the primitive patient with a rigged 3D human model and hand animations.
- Add compression-depth measurement and chest recoil scoring.
- Add audio metronome and sound feedback for the 100-120 BPM target.
- Add AED training and rescue-breath stages.
- Add difficulty levels and different emergency scenarios.
- Save multiple player profiles and a leaderboard.
- Add analytics showing average BPM, accuracy, and common mistakes.

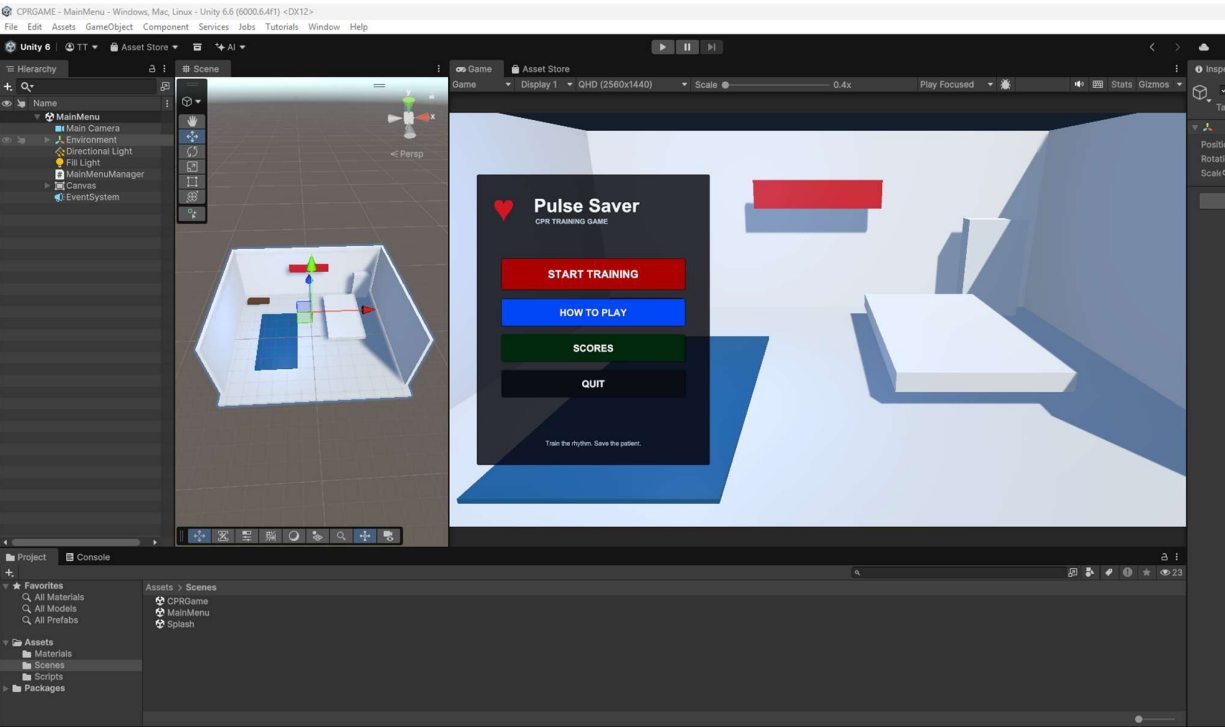
15. Conclusion

Pulse Saver converts CPR timing into an interactive Unity game with clear visual feedback and measurable performance. The core mechanic is based on the interval between chest-compression inputs, which is converted to BPM and compared with the 100-120 BPM target. Accuracy, patient health, progress, and streak are combined into a score, while the final success condition determines whether the patient is saved or dies. The project also includes persistent JSON score storage and a fully editable scene-based GUI, making it suitable for further development and demonstration.

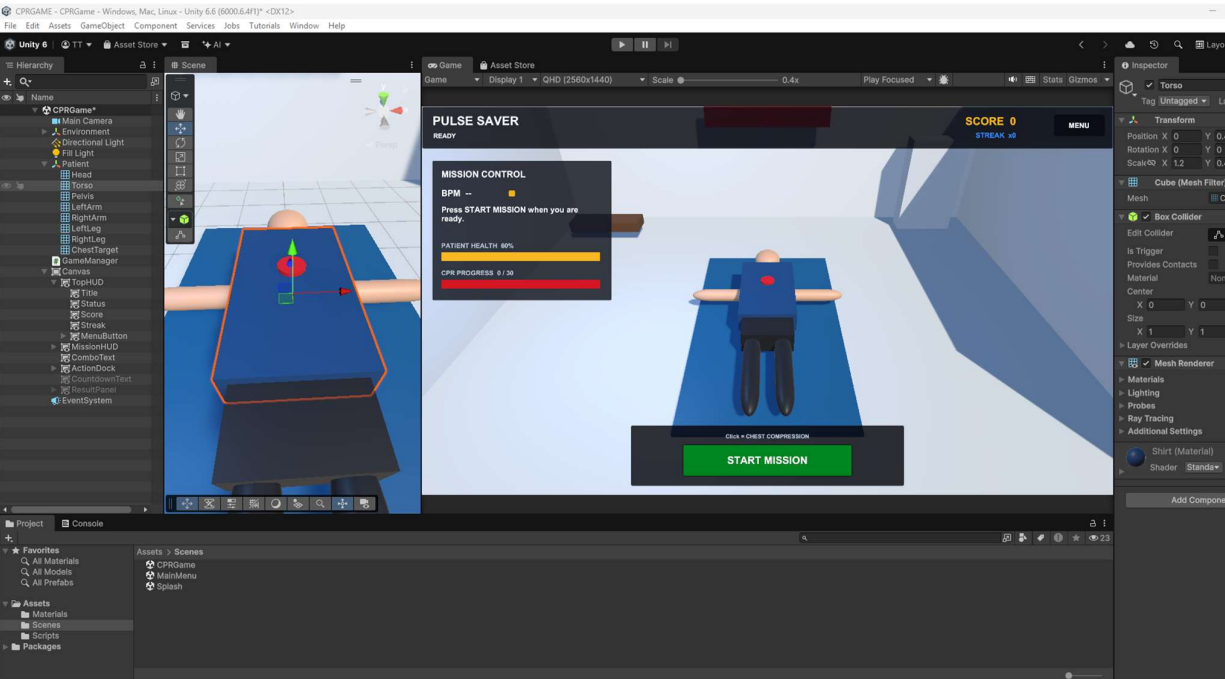
Splash Scene:



Main Menu Scene:



CPR Training Scene:



GamePlay:

